

Analysis & Complexity

Recursive Algorithm

1) Time Complexity Analysis :

- **isTriangle** : Performs a constant number of comparisons
if $(a + b > c)$ AND $(a + c > b)$ AND $(b + c > a)$ => (Three conditions)
, Time Complexity = $O(1)$
- **Merge** : The main operation in the merge function is inserting elements into the output array `arr[k]`. Each element from the two subarrays => `leftArr (size n1)` and `rightArr (size n2)` is processed exactly once during copying and merging. The total number of processed elements is $n_1 + n_2 = n$, Time Complexity = $O(n)$
- **MergeSort** : The algorithm follows the divide-and-conquer paradigm, where the array is recursively divided into two halves until single elements are reached , resulting in approximately $\log(n)$ levels of recursion. At each level, all n elements are processed during the merging step , leading to n operations per level. The recurrence relation is given by $T(n) = 2T(n/2) + O(n)$. Solving this recurrence results in a Time Complexity of $O(n \log n)$.
- **RecursiveSolution**: The function checks consecutive triplets and increments the index by one in each recursive call. In the worst case, it evaluates all possible triplets, which are $(n - 2)$ => Time Complexity = $O(n)$

Main Operations :

- Sorting The array using merge sort algorithm $\Rightarrow O(n \log n)$
- Checking for a valid triangle using recursive solution algorithm $\Rightarrow O(n)$

Total Time Complexity:

- $T(n) = \text{Sorting } O(n \log n) + \text{Checking } O(n)$
- $T(n) = O(n \log n) + O(n) = O(n \log n) \Rightarrow$ Since $O(n \log n)$ grows faster than $O(n)$, it dominates .

- Both Best and Worst Case are $O(n \log n)$ because Merge Sort is the dominant factor. Even if the recursive check finds a triangle immediately ($O(1)$), the total complexity remains the same $\Rightarrow T(n) = O(n \log n) + O(1) = O(n \log n)$

2) Space Complexity Analysis :

- **Auxiliary Space** : Requires additional memory for temporary arrays `leftArr` and `rightArr` during the merging process. These arrays are created and freed after each merge operation, and new temporary arrays are allocated for following merges $\Rightarrow$ Space Complexity = $O(n)$
- **Recursion Stack** : The depth of recursion in MergeSort is proportional to $\log(n)$, as the problem size is halved at each recursive call $\Rightarrow$ Space Complexity = $O(\log n)$

Total Space Complexity

- $S(n) = \text{Auxiliary Space } O(n) + \text{Recursion Stack } O(\log n)$
- $S(n) = O(n) + O(\log n) = O(n) \Rightarrow$ Since $O(n)$ grows faster than $O(\log n)$, it dominates .